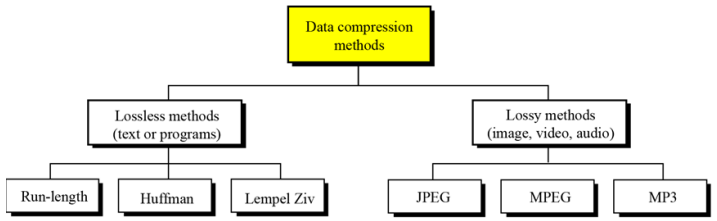
SCC.160: Fundamentals of Communication Systems

Lecture 3

Hassan Ahmed
h.ahmed@lancaster.ac.uk

1

Data compression methods



2

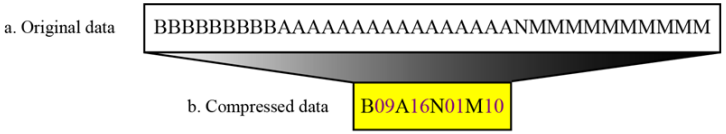
Run-length encoding

- Run-length encoding is probably the simplest method of compression. It can be used to compress data made of any combination of symbols. It does not need to know the frequency of occurrence of symbols and can be very efficient if data is represented as 0s and 1s.
- The general idea behind this method is to replace consecutive repeating occurrences of a symbol by one occurrence of the symbol followed by the number of occurrences.

3

Cont'd

- The method can be even more efficient if the data uses only two symbols (for example 0 and 1) in its bit pattern and one symbol is more frequent than the other.



4

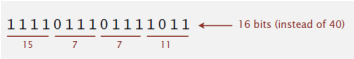
Run-length encoding for two symbols

Simple type of redundancy in a bitstream. Long runs of repeated bits.

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1

Representation. 4-bit counts to represent alternating runs of 0s and 1s:

15 0s, then 7 1s, then 7 0s, then 11 1s.



Applications. JPEG, ITU-T T4 Group 3 Fax, ...

5

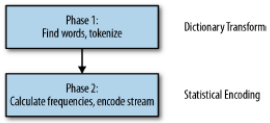
Lempel Ziv encoding

- Lempel Ziv (LZ) encoding is an example of a category of algorithms called dictionary-based encoding. The idea is to create a dictionary (a table) of strings used during the communication session.
- If both the sender and the receiver have a copy of the dictionary, then previously-encountered strings can be substituted by their index in the dictionary to reduce the amount of information transmitted.

6

Cont'd

- Dictionary transforms take a source stream, first construct some dictionary of words (rather than symbols), and then apply statistical compression based on the words in the dictionary.
- Dictionary transforms are not meant to be a replacement for statistical encoding, but rather a transform that we first apply to our stream, so that it can be encoded more effectively by a statistical encoder



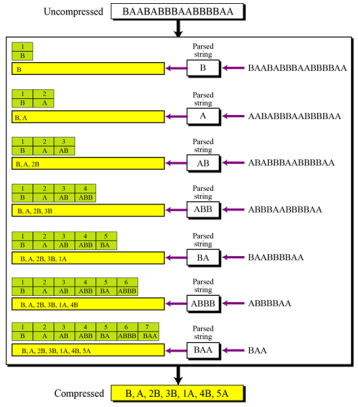
7

Compression

- In this phase there are two concurrent events: building an indexed dictionary and compressing a string of symbols.
- The algorithm extracts the smallest substring that cannot be found in the dictionary from the remaining uncompressed string.
- It then stores a copy of this substring in the dictionary as a new entry and assigns it an index value.
- Compression occurs when the substring, except for the last character, is replaced with the index found in the dictionary. The process then inserts the index and the last character of the substring into the compressed string.

8

An example of Lempel Ziv encoding



Lancaster University

9

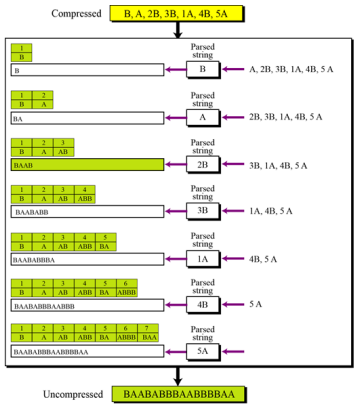
Decompression

Lancaster University

- Decompression is the inverse of the compression process.
- The process extracts the substrings from the compressed string and tries to replace the indexes with the corresponding entry in the dictionary, which is empty at first and built up gradually.
- The idea is that when an index is received, there is already an entry in the dictionary corresponding to that index.

10

An example of Lempel Ziv decoding



Lancaster University

11

Lancaster University

Lossy Compression Methods

12

LOSSY COMPRESSION METHODS

- Loss of information is not acceptable in a text file or a program file.
- It is, however, acceptable in an image, video, or audio file.
- The reason is that our eyes and ears cannot distinguish subtle changes. In such cases, we can use a **lossy data compression** method.
  - Can compress complex images 10:1 to 50:1 and retain high quality, and 100 to 200 times for lower quality, but acceptable, images
- These methods are cheaper—they take less time and space when it comes to sending millions of bits per second for images and video.

13

Image Compression

- Compression algorithms are developed by taking advantage of the redundancy that is inherent in image data

✓ Four primary types of redundancy that can be found in images are:

1. *Coding*
2. *Interpixel*
3. *Interband*
4. *Psychovisual redundancy*

14

Redundancies in Image

1. *Coding redundancy*

✓ Occurs when the data used to represent the image is not utilized in an optimal manner

2. *Interpixel redundancy*

✓ Occurs because adjacent pixels tend to be highly correlated, in most images the brightness levels do not change rapidly, but change gradually

15

Cont'd

3. *Interband redundancy*

✓ Occurs in color images due to the correlation between bands within an image – if we extract the red, green and blue bands they look similar

4. *Psychovisual redundancy*

✓ Some information is more important to the human visual system than other types of information

16

Coding - recap

- Code: a list of symbols (letters, numbers, bits etc.)
- Code word: a sequence of symbols used to represent some information (e.g., gray levels).
- Code word length: number of symbols in a code word.

Example: (binary code, symbols: 0,1, length: 3)

0: 000	4: 100
1: 001	5: 101
2: 010	6: 110
3: 011	7: 111

17

Coding - Definitions (cont'd)

N x M image

r_k: k-th gray level

$l(r_k)$: # of bits for r_k

$P(r_k)$: probability of r_k

Average # of bits: $L_{avg} = E(l(r_k)) = \sum_{k=0}^{L-1} l(r_k)P(r_k)$

Total # of bits: NML_{avg}

18

Coding Redundancy

- Case 1: $l(r_k)$ = constant length

r_k	$p_r(r_k)$	Code 1	$l_1(r_k)$
$r_0 = 0$	0.19	000	3
$r_1 = 1/7$	0.25	001	3
$r_2 = 2/7$	0.21	010	3
$r_3 = 3/7$	0.16	011	3
$r_4 = 4/7$	0.08	100	3
$r_5 = 5/7$	0.06	101	3
$r_6 = 6/7$	0.03	110	3
$r_7 = 1$	0.02	111	3

Assume an image with $L = 8$

Assume $l(r_k) = 3$, $L_{avg} = \sum_{k=0}^{7} 3P(r_k) = 3\sum_{k=0}^{7} P(r_k) = 3$ bits

Total number of bits: $3NM$

19

Coding Redundancy (cont'd)

- Case 2: $l(r_k)$ = variable length

Table 6.1 Variable-Length Coding Example — variable length

r_k	$p_r(r_k)$	Code 1	$l_1(r_k)$	Code 2	$l_2(r_k)$
$r_0 = 0$	0.19	000	3	11	2
$r_1 = 1/7$	0.25	001	3	01	2
$r_2 = 2/7$	0.21	010	3	10	2
$r_3 = 3/7$	0.16	011	3	001	3
$r_4 = 4/7$	0.08	100	3	0001	4
$r_5 = 5/7$	0.06	101	3	00001	5
$r_6 = 6/7$	0.03	110	3	000001	6
$r_7 = 1$	0.02	111	3	000000	6

Total number of bits: 2.7NM

$L_{avg} = \sum_{k=0}^{7} l(r_k)P(r_k) = 2.7$ bits

$C_R = \frac{3}{2.7} = 1.11$ (about 10%)

$R_D = 1 - \frac{1}{1.11} = 0.099$

20

Psychovisual redundancy

- The human eye is more sensitive to the lower frequencies than to the higher frequencies in the visual spectrum.
- Idea: discard data that is perceptually insignificant!

21

Example: quantization

256 gray levels

16 gray levels

16 gray levels + random noise

C=8/4 = 2:1

Add a small pseudo-random number to each pixel prior to quantization

22

Measuring Information

- A key question in image compression is:
  - "What is the minimum amount of data that is sufficient to describe completely an image without loss of information?"
- How do we measure the information content of an image?

23

Measuring Information (cont'd)

- We assume that information generation is a probabilistic process.
- Idea: associate information with probability!
- A random event E with probability $P(E)$ contains:

$$I(E) = log(\frac{1}{P(E)}) = -log(P(E)) \text{ units of information}$$

- Note: I(E)=0 when P(E)=1

24

How much information does a pixel value contain?

- Suppose that gray level values are generated by a random process, then r_k contains:

$$I(r_k) = -\log(P(r_k))$$ units of information!

(assume statistically independent random events)

25

How much information does an image contain?

- Average information content of an image:

$$E = \sum_{k=0}^{L-1} I(r_k)\,\mathrm{P}(r_k)$$

using $I(r_k) = -\log(P(r_k))$

Entropy: $H = -\sum_{k=0}^{L-1} P(r_k)\log(P(r_k))$ units/pixel (e.g., bits/pixel)

26

Redundancy

- **Redundancy:** (data vs info) $R = L_{avg} - H$

where: $L_{avg} = E(l(r_k)) = \sum_{k=0}^{L-1} l(r_k)P(r_k)$

Note: if $L_{avg} = H$, then R=0 (no redundancy)

27

Entropy Estimation

21	21	21	95	169	243	243	243
21	21	21	95	169	243	243	243
21	21	21	95	169	243	243	243
21	21	21	95	169	243	243	243

Gray Level	*Count*	*Probability*
21	12	3/8
95	4	1/8
169	4	1/8
243	12	3/8

First order estimate of H:

$$H = -\sum_{k=0}^{3} P(r_k)\log(P(r_k)) = 1.81 \text{ bits/pixel}$$

L_{avg} = 8 bits/pixel R= L_{avg}- H

The first-order estimate provides only a lower-bound on the compression that can be achieved.

28

Second order estimate of H:

- Use relative frequencies of pixel blocks :

21	21	21	95	169	243	243	243
21	21	21	95	169	243	243	243
21	21	21	95	169	243	243	243
21	21	21	95	169	243	243	243

Gray Level Pair	*Count*	*Probability*
(21, 21)	8	1/4
(21, 95)	4	1/8
(95, 169)	4	1/8
(169, 243)	4	1/8
(243, 243)	8	1/4
(243, 21)	4	1/8

$$H = -\sum_{k=0}^{L-1} P(r_k) log(P(r_k))$$

$$H = 2.5/2 = 1.25 \text{ bits/pixel}$$

29

Differences in Entropy Estimates

- Differences between higher-order estimates of entropy and the first-order estimate indicate the presence of interpixel redundancy!
- Need to apply some transformation to deal with interpixel redundancy!

30

Differences in Entropy Estimates (cont'd)

- Example: consider pixel differences

21	21	21	95	169	243	243	243
21	21	21	95	169	243	243	243
21	21	21	95	169	243	243	243
21	21	21	95	169	243	243	243

21	0	0	74	74	74	0	0
21	0	0	74	74	74	0	0
21	0	0	74	74	74	0	0
21	0	0	74	74	74	0	0

Gray Level or Difference	*Count*	*Probability*
0	16	1/2
21	4	1/8
74	12	3/8

31

Cont'd

- What is the entropy of the pixel differences image?

$$H = -\sum_{k=0}^{2} P(r_k) log(P(r_k)) = 1.41 \text{ bits/pixel}$$

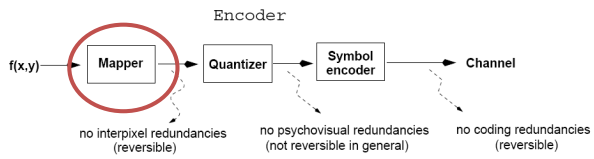
(better than the entropy of the original image H=1.81)

- An even better transformation should be possible since the second order entropy estimate is lower:

$$1.41 \text{ bits/pixel} > 1.25 \text{ bits/pixel}$$

32

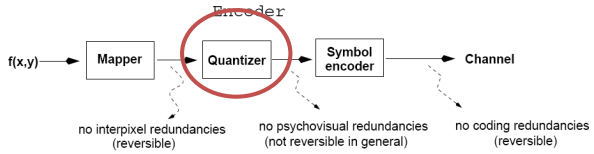
Image Compression Model



- **Mapper**: transforms data to account for interpixel redundancies.

33

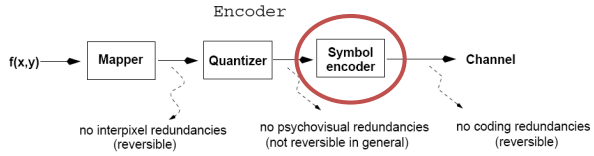
Image Compression Model



- **Quantizer**: quantizes the data to account for psychovisual redundancies.

34

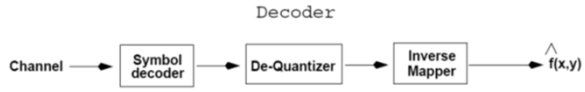
Image Compression Model



- **Symbol encoder**: encodes the data to account for coding redundancies.

35

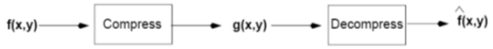
Image Compression Models (cont'd)



The decoder applies the inverse steps.

36

Fidelity Criteria



- How close is $f(x,y)$ to $\hat{f}(x,y)$? ?
- Criteria
  - Subjective: based on human observers
  - Objective: mathematically defined criteria

37

Subjective Fidelity Criteria

Value	Rating	Description
1	Excellent	An image of extremely high quality, as good as you could desire.
2	Fine	An image of high quality, providing enjoyable viewing. Interference is not objectionable.
3	Passable	An image of acceptable quality. Interference is not objectionable.
4	Marginal	An image of poor quality; you wish you could improve it. Interference is somewhat objectionable.
5	Inferior	A very poor image, but you could watch it. Objectionable interference is definitely present.
6	Unusable	An image so bad that you could not watch it.

38

Objective Fidelity Criteria

- Root mean square error (RMS)

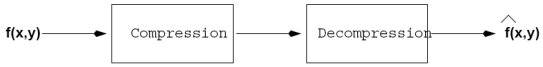
$$e_{rms} = \sqrt{\frac{1}{MN}\sum_{x=0}^{M-1}\sum_{y=0}^{N-1}(\hat{f}(x,y) - f(x,y))^2}$$

- Mean-square signal-to-noise ratio (SNR)

$$SNR_{ms} = \frac{\sum_{x=0}^{M-1}\sum_{y=0}^{N-1}(\hat{f}(x,y))^2}{\sum_{x=0}^{M-1}\sum_{y=0}^{N-1}(\hat{f}(x,y) - f(x,y))^2}$$

39

Lossless Compression

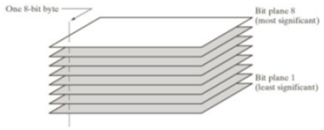


$$e(x,y) = \hat{f}(x,y) - f(x,y) = 0$$

40

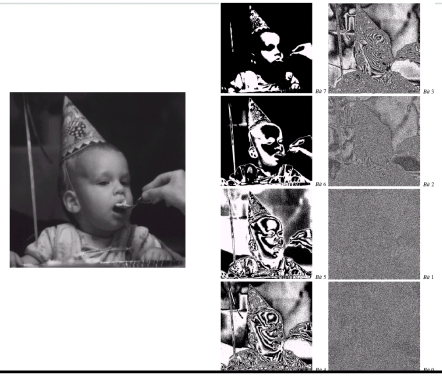
Bit-Plane Coding

- An effective technique to reduce inter pixel redundancy is to process each bit plane individually
- The image is decomposed into a series of binary images.
- Each binary image is compressed using one of well known binary compression techniques



41

Bit-Plane Decomposition



42

Jpeg Compression

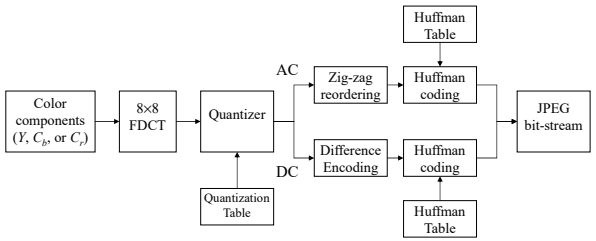
43

JPEG Compression

- JPEG, which stands for Joint Photographic Experts Group (the name of the committee that created the JPEG standard) is a lossy compression algorithm for images.
- The JPEG algorithm takes advantage of the fact that humans can't see colours at high frequencies. These high frequencies in the image that are eliminated during the compression. JPEG compression also works best on images with smooth colour transitions.

44

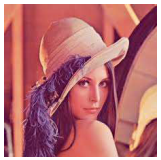
JPEG Compression



45

Splitting into 8x8

- The input image is divided into smaller blocks having 8 x 8 dimensions, summing up to 64 pixel in total.



Original image

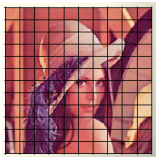


Image split into multiple 8x8 pixel blocks

46

亮度 色度

Colour space conversion (RGB to YCbCr)

- For color signals JPEG transforms the RGB components into the *YCbCr* color space, and subsamples the two color difference signals (*Cr* and *Cb*), since we perceive more detail in the luminance (brightness) than in the color information

$$\begin{bmatrix} Y \\ C_b \\ C_r \end{bmatrix} = \begin{bmatrix} 0.299000 & 0.587000 & 0.114000 \\ -0.168736 & -0.331264 & 0.500002 \\ 0.500000 & -0.418688 & -0.081312 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} 0 \\ 128 \\ 128 \end{bmatrix}$$

(a) translate from RGB to YC_bC_r

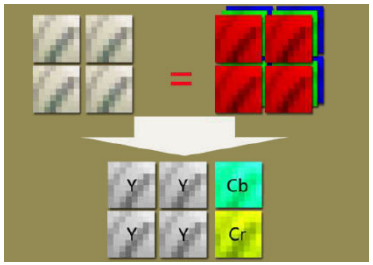
$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1.0 & 0.0 & 1.40210 \\ 1.0 & -0.34414 & -0.71414 \\ 1.0 & 1.77180 & 0.0 \end{bmatrix} \begin{bmatrix} Y \\ C_b - 128 \\ C_r - 128 \end{bmatrix}$$

(b) translate from YC_bC_r to RGB

47

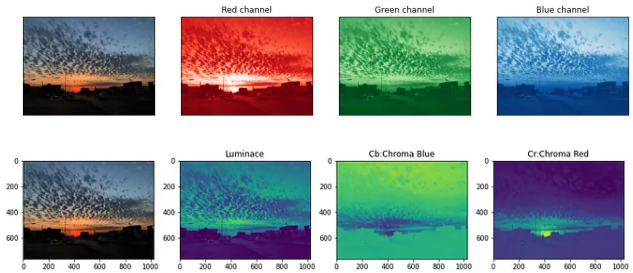
Down-sampling

- Y is taken for every pixel, and Cb, Cr are taken for a block of 2x2 pixels.



48

Colour space conversion



49

Forward DCT 将8x8的像素块从空间域转换到频域

- The DCT uses the cosine function, therefore not interacting with complex numbers at all.
- DCT converts the information contained in a block(8x8) of pixels from ***spatial*** domain to the ***frequency*** domain.
  - Human vision is insensitive to high frequency components, due to which it is possible to treat the data corresponding to high frequencies as redundant

$$F(u,v)=\frac{1}{4}C(u)C(v)\sum_{x=0}^{7}\sum_{y=0}^{7}f(x,y)\cos\left[\frac{\pi(2x+1)u}{16}\right]\cos\left[\frac{\pi(2y+1)v}{16}\right]$$

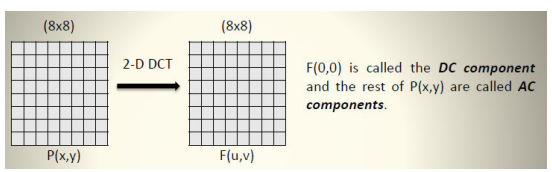
for $u=0,\ldots,7$ and $v=0,\ldots,7$

where $C(k)=\begin{cases}1/\sqrt{2} & \text{for } k=0\\ 1 & \text{otherwise}\end{cases}$

50

DC and AC components

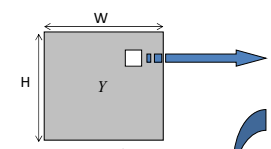
- For $u = v = 0$ the two cosine terms are 0 and hence the value in the location *F[0,0]* of the transformed matrix is simply a function of the summation of all the values in the input matrix.
- This is ***the mean*** of all 64 values in the matrix and is known as the **DC coefficient.**
- Since the values in all the other locations of the transformed matrix have a frequency coefficient associated with them they are known as **AC coefficients.**



51

Forward DCT

Example :



the luminance of an image

48	39	40	68	60	38	50	121
149	82	79	101	113	106	27	62
58	63	77	69	124	107	74	125
80	97	74	54	59	71	91	66
18	34	33	46	64	61	32	37
149	108	80	106	116	61	73	92
211	233	159	88	107	158	161	109
212	104	40	44	71	136	113	66

8x8 values of luminance

DCT

699.25	43.18	55.25	72.11	24.00	-25.51	11.21	-4.14
-129.78	-71.50	-70.26	-73.35	59.43	-24.02	22.61	-2.05
85.71	30.32	61.78	44.87	14.84	17.35	15.51	-13.19
-40.81	10.17	-17.53	-55.81	30.50	-2.28	-21.00	-1.26
-157.50	-49.39	13.27	-1.78	-8.75	22.47	-8.47	-9.23
92.49	-9.03	45.72	-48.13	-58.51	-9.01	-28.54	10.38
-53.09	-62.97	-3.49	-19.62	56.09	-2.25	-3.28	11.91
-20.54	-55.90	-20.59	-18.19	-26.58	-27.07	8.47	0.31

52

Quantization

Lancaster University

DCT系数

$$F_q(u,v) = Round\left(\frac{F(u,v)}{Q(u,v)}\right)$$

量化矩阵

- F(u,v) represents a *DCT coefficient*, Q(u,v) is *quantization matrix*, and F_q (u, v) represents the *quantized DCT coefficients* to be applied for successive entropy coding.
- The quantization step is the major information losing step in JPEG compression.
- For non-uniform quantization, there are 2 psycho-visually tuned quantization tables each for luminance (Y) and chrominance (Cb,Cr) components defined by jpeg standards.

Quantization (Cont'd)

Lancaster University

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

The Luminance Quantization Table

17	18	24	47	99	99	99	99
18	21	26	66	99	99	99	99
24	26	56	99	99	99	99	99
47	66	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99

The Chrominance Quantization Table

- The entries of Q(u,v) tend to have larger values towards the lower right corner. This aims to introduce more loss at the higher spatial frequencies.
- The tables above show the default Q(u,v) values obtained from psychophysical studies with the goal of maximizing the compression ratio while minimizing perceptual losses in JPEG images.

Quantization

Lancaster University

- Each element of F(u,v) is divided by the corresponding element of Q(u,v) and then rounded off to the nearest integer to get the F_q(u,v) matrix.

699.25	43.18	55.25	72.11	24.00	-25.51	11.21	-4.14
-129.78	-71.50	-70.26	-73.35	59.43	-24.02	22.61	-2.05
85.71	30.32	61.78	44.87	14.84	17.35	15.51	-13.19
-40.81	10.17	-17.53	-55.81	30.50	-2.28	-21.00	-1.26
-157.50	-49.39	13.27	-1.78	-8.75	22.47	-8.47	-9.23
92.49	-9.03	45.72	-48.13	-58.51	-9.01	-28.54	10.38
-53.09	-62.97	-3.49	-19.62	56.09	-2.25	-3.28	11.91
-20.54	-55.90	-20.59	-18.19	-26.58	-27.07	8.47	0.31

$F(u,v)$

Quantizer

$Q(u,v)$

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

43.70	3.93	5.52	4.51	1.00	-0.64	0.22	-0.07
-10.82	-5.96	-5.02	-3.86	2.29	-0.41	0.38	-0.04
6.12	2.33	3.86	1.87	0.37	0.30	0.22	-0.24
-2.91	0.60	-0.80	-1.92	0.60	-0.03	-0.26	-0.02
-8.75	-2.25	0.36	-0.03	-0.13	0.21	-0.08	-0.12
3.85	-0.26	0.83	-0.75	-0.72	-0.09	-0.25	0.11
-1.08	-0.98	-0.04	-0.23	0.54	-0.02	-0.03	0.12
-0.29	-0.61	-0.22	-0.19	-0.24	-0.27	0.08	0.00

$$\frac{F(u,v)}{Q(u,v)}$$

Quantization

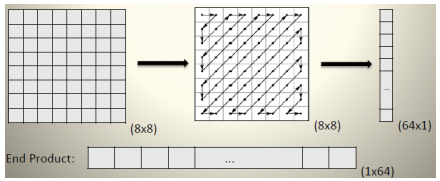
Lancaster University

$$F_q(u,v) = Round\left(\frac{F(u,v)}{Q(u,v)}\right)$$

44	4	6	5	1	-1	0	0
-11	-6	-5	-4	2	0	0	0
6	2	4	2	0	0	0	0
-3	1	-1	-2	1	0	0	0
-9	-2	0	0	0	0	0	0
4	0	1	-1	-1	0	0	0
-1	-1	0	0	1	0	0	0
0	-1	0	0	0	0	0	0

Zig-Zag Reordering (AC)

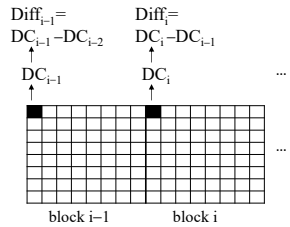
- Maps 8 x 8 matrix to a 1 x 64 vector.
- Why zigzag scanning?
- – To group low frequency coefficients at the top of the vector and high frequency coefficients at the bottom.
- In order to exploit the presence of the large number of zeros in the quantized matrix, a zigzag of the matrix is used.



57

DPCM on DC component

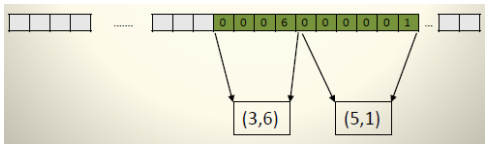
- Differential Pulse Code Modulation (DPCM) is applied to the DC component.
- DC component is large and varies, but often close to previous value.
- DPCM encodes the difference between the current and previous 8x8 block.



58

RLE on AC components

- Run Length Encoding (RLE) is applied to the AC components.
- 1x63 vector (AC) has lots of zeros in it.
- Encoded as (***skip, value***) pairs, where *skip* is the number of zeros preceding a non-zero value in the quantized matrix, and *value* is the actual coded value of the non-zero component.
- (0,0) is sent as end-of-block (EOB).



59

Huffman Coding

- DC and AC components finally need to be represented by a smaller number of bits
- Why Huffman Coding?
  - Significant levels of compression can be obtained by replacing long strings of binary digits by a string of much shorter code words.
- How?
  - The length of each code word is a function of its relative frequency of occurrence.
- Normally a table of code words is used with the set of code words precomputed using the Huffman Coding Algorithm.
- In Huffman Coding, each DPCM-coded DC coefficient is represented by a pair of symbols : *(Size, Amplitude)*
- where *Size* indicates number of bits needed to represent the coefficient and *Amplitude* contains actual bits.

60

Huffman Coding: DC Components

Lancaster University

- DC Components are coded as (*Size, Value*). The look-up table for generating code words for Value is as given below:-

Size	Values	Bits for the value
1	-1,1	0,1
2	-3,-2,2,3	00,01,10,11
3	-7,-6,-5,-4,4,5,6,7	000,001,010,011,100,101,110,111
4	-15,...,-8,8,...,15	0000,...,0111,1000,...,1111
5	-31,...,-16,16,...31	00000,...,01111,10000,...,11111
6	-63,...,-32,32,...63	000000,...,011111,100000,...,111111
7	-127,...,-64,64,...,127	0000000,...,0111111,1000000,...,1111111
8	-255,..,-128,128,..,255	...
9	-511,..,-256,256,..,511	...
10	-1023,..,-512,512,..,1023	...
11	-2047,..,-1024,1024,..,2047	...

61

Huffman DC Coding - Example

Lancaster University

- If the DC component is 48, and the previous DC component is 52. Then the difference between the 2 components is (48-52) = -4.
- Therefore it is Huffman coded as: 100 011
- 011: The codes for representing -4 (Table on previous slide)
- 100: The size of the value code word is 3.

SIZE	Code Length	Code
0	2	00
1	3	010
2	3	011
3	3	100
4	3	101
5	3	110
6	4	1110
7	5	11110
8	6	111110
9	7	1111110
10	8	11111110
11	9	111111110

62

Run Length Coding – AC Component

Lancaster University

- Example 1

- 63 AC coefficience:
- 57, 45, 0, 0, 0, 0, 23, 0, -30, -16, 0, 0, 1, $\underbrace{0, 0, 0, 0, 0, 0, 0, 0,\ldots, 0}_{\text{50 zeros}}$
- Run Length Coding :
- (0,57) ; (0,45) ; (4,23) ; (1,-30) ; (0,-16) ; (2,1) ; EOB → (0,0)
- Encode the right value of these pair as category and bits for the value, except the special markers like (0,0) or (15,0) :

  (0,6,111001) ; (0,6,101101) ; (4,5,10111);

  (1,5,00001) ; (0,4,0111) ; (2,1,1) ; (0,0)

63

Huffman coding

Lancaster University

run/category	code length	code word
0/0	4	1010
...		
0/6	7	1111000
...		
0/10	16	1111111110000011
1/1	4	1100
...		
1/5	11	11111110110
...		
4/5	16	1111111110011000
...		
15/10	16	1111111111111110

64

Huffman coding

- Example

The AC coefficiences :
(0,6,111001) ; (0,6,101101) ; (4,5,10111);
(1,5,00001) ; (0,4,0111) ; (2,1,1) ; (0,0)

Encode the left two value in () using Huffman encoding :

- 1111000 111001 , 1111000 101101 , 1111111110011000 10111 ,
- 11111110110 00001 , 1011 0111 , 11100 1 , 1010

65

RLE

original bit stream: 000000111111111111110000000000000111111111 (42 bits)

a run of 6 0-bits *a run of 14 1-bits*

Representing each run in the original bit stream as a pair **b : n** (where b is 0 or 1 to indicate the contents of the run, and n is the length of the run) produces:

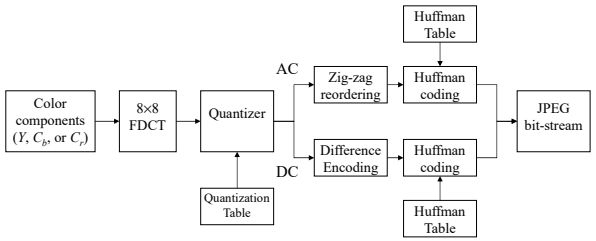
sequence of runs: 0:6, 1:14, 0:13, 1:9

resulting 5-bit bytes: 00110, 11110, 01101, 11001

compressed bit stream: 00110111100110111001 (20 bits)

66

JPEG Compression



67

Thank you

68